

CSC360 Final Project: Insertion Sort and Binary Search Algorithm Implementation in RISC-V Assembly

Mark Stotzer*, Dawson Blakeslee**

Hamm School of Engineering at the University of Mary, Bismarck, ND

*mpstotzer1@umary.edu, **diblakeslee1@umary.edu

Abstract—

Index Terms—

I. INTRODUCTION

HERE is an example of how to cite [1].

II. BACKGROUND AND SCOPE OF PROBLEM

III. DESIGN APPROACH

A. Insertion Sort

The design process for the insertion sort algorithm began with research into how the algorithm functioned, as to best understand the problem, solution, and implementation. Next, a high-level flowchart was developed from the methods found during the research stage, and can be found in Figure 1. From there, C code was made to accomplish the insertion sort task, which could then be more easily translated to RISC-V assembly. After the C implementation, a low-level flowchart (Figure 2) was created, which proved very useful in the proceeding step-by-step translation into assembly. After the translation to assembly, the program was tested in Ripes to verify completeness and to observe certain metrics provided by Ripes. The C program was then translated once again into assembly, but this time using an optimized 'gcc' style compiler. Finally, the optimized compiler program was compared to the hand-generated program in Ripes to document improvements in CPI, clock cycles, instruction count, and execution time.

B. Binary Search

For simple algorithms like binary search, the first step was drawing a high-level flowchart (see figure 3). Then, it was converted into C code and tested for bugs. This was converted into a low-level flowchart (figure 4) and then, given the simplicity of the algorithm, converted into assembly by hand to produce the naive implementation. An optimized compiler-generated algorithm, referred to as just the compiler algorithm, followed, and both were tested in Ripes. Finally, after studying

the hazards, they were modified into one final algorithm, called the modified algorithm.

The algorithms were tested by averaging the cycles it took to find an element in the deepest level of the tree and two deepest level searches for items not in the tree (too low and too high, respectively). This produced the results seen in figure 7.

IV. FLOWCHARTS AND DESIGN PROGRESSION

A. Insertion Sort

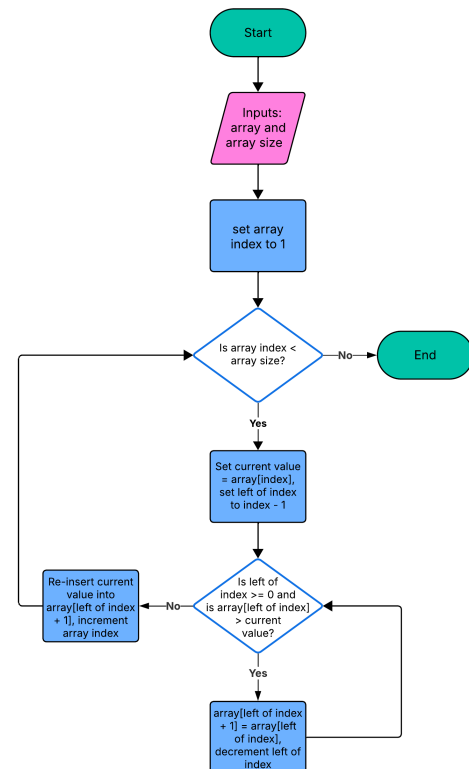


Fig. 1. High-Level Insertion Sort Flowchart

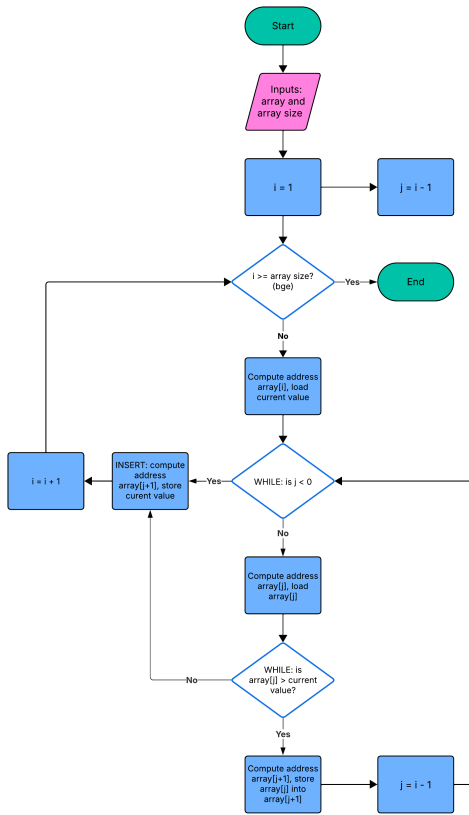


Fig. 2. Low-Level Insertion Sort Flowchart

B. Binary Search

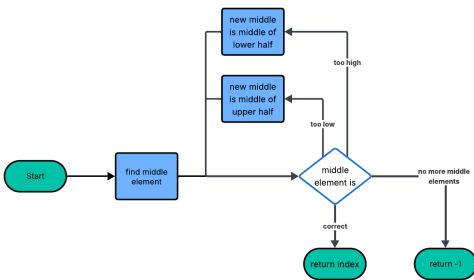


Fig. 3. High-Level Binary Search Flowchart

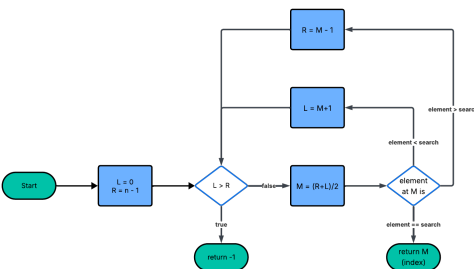


Fig. 4. Low-Level Binary Search Flowchart

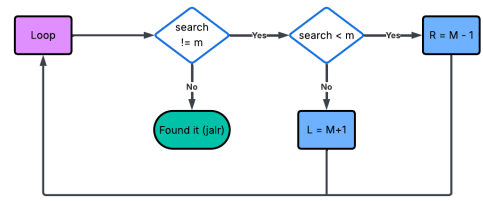


Fig. 5. Non-Pipelined Naive Branching

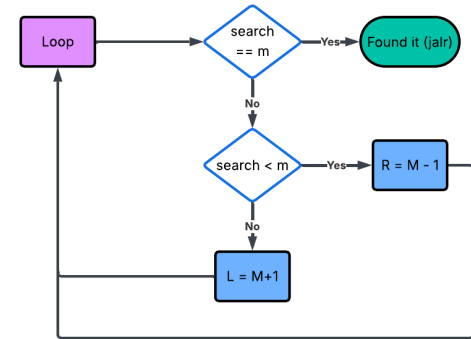


Fig. 6. Pipelined Modified Branching

V. PIPELINING AND HAZARDS

A. Insertion Sort

B. Binary Search

Looking at the different algorithm diagrams, there are two hazards. First is a load-use hazard "lw x31, 0, x31" followed by a branch statement using x31. This is common to all three since the code could not be rearranged to place this earlier.

The second hazard is a branch hazard rooted in the arrangement of branch statements in the assembly code. In the naive algorithm (Fig. 5), one branch statement jumped to the next upon success in a daisy chain pattern. This caused the two pipelined instructions after to be flushed upon failure. Since there were three cases, the chance of any branch failing was higher than those of it succeeding. There was an opportunity for three branch hazards per loop. Both the modified and compiler algorithms prevented this by placing all branch statements adjacent to each other in the code (see Fig. 6). As such, a failure would fall through to evaluating the next branch, causing at most one branch hazard per loop.

VI. ASSEMBLY IMPLEMENTATION

A. Insertion Sort

B. Binary Search

There were three different assembly implementations: the naive, the compiler, and the modified. The naive and modified are entirely the same except for the order of branching (see

section V for details). The compiler version has the same branch ordering trick as the modified, but included two few extra instructions on top. First, it set the return register to -1 by default, saving a cycle later if the item was not found but wasting a cycle if it was. Second, it immediately checked if the array length was greater than zero, saving time for an invalid array parameter but wasting time on valid ones.

REFERENCES

[1]
[2]

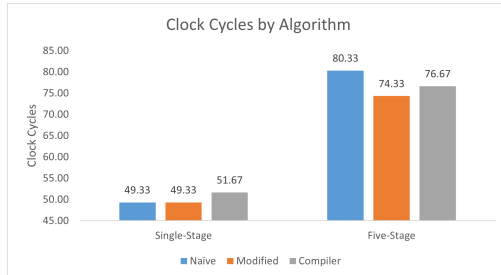


Fig. 7. Performance by Algorithm and Processor

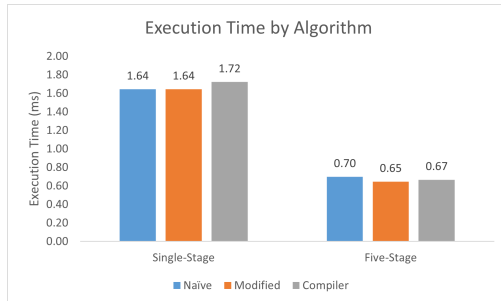


Fig. 8. Execution Time by Algorithm and Processor

VII. RESULTS

A. Insertion Sort

B. Binary Search

The fastest algorithm, for both processors, was the modified algorithm. It tied with the naive in single-stage, given that there is no branch penalty, and beat the compiler by approximately two cycles. In the five-stage, the branch arrangement for modified and compiler easily beat out naive. Still, the compiler's extra introductory steps served to slow it down by 2 cycles.

It should be noted that single stage took half the cycles of five-stage, because the latter must flush and stall frequently. However, the advantage is that the five-stage runs five cycles in the time the single-stage runs one. In practice, due to added hardware complexity for pipelining, the clock period is only around three times as fast. A standard clock period for RISC-V hardware was found online: 30 and 115 MHz for single and five-stage respectively. The estimated execution times, seen in figure 8, demonstrate the same pattern identified above: modified is fastest.

VIII. CONCLUSION